Electronics & Computing Club

Simon Says Project

Build & Code: Arduino Workshop

Written by Mark Drobyshev and Alexander D'Alesio on February 8th, 2025

Section 1
Overview

Have you ever played those Simon Says games with the four colors? Well now you can take a deep dive into how the code functions within these toys and plays out to create the classic Simon Says. This project will allow you to work with the concept of randomization and provide a handle on how to assemble electronic circuits. With a few buttons, LEDs and wires, and code, you'll have made your very own game. Remember, the more sequences you enter correctly, the longer and harder the game becomes, so I hope you have some sharp eyes and good memory!

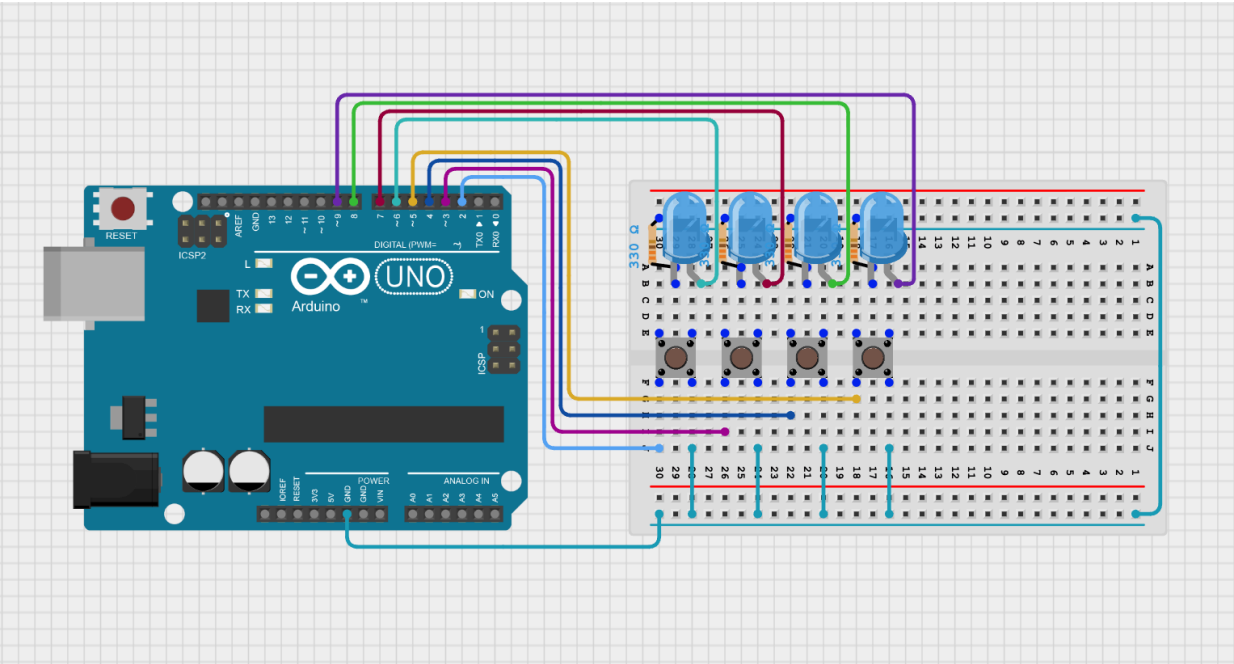
Section 2
Required Components

Here's a list of what you'll need:

- 4x Pushbuttons
- 4x 330Ω Resistors
- 4x LEDs
- 1x Breadboard
- 1x Arduino Uno
- Assorted Breadboard Wires
- Assorted Jumper Wires

Section 3

Circuit Assembly



https://app.cirkitdesigner.com/project/23ec04c3-6741-4997-98b5-2106fd99e6b2

Please refer to the image above for the complete circuit wiring. A link to the online schematic can be found under the image in case you need higher resolution.

If this is your first time using a breadboard (the white rectangle in the image with many holes), please refer to our *Breadboard Guide*.

1. **Connect the Ground Bus Strips Together**
   The two horizontal ground bus strips should be interconnected, as shown in the diagram on the previous page (see the blue wire on the right-hand side of the breadboard). After bridging the bus strips, use a wire to connect either the top or bottom bus strip to one of the Arduino's ground pins.

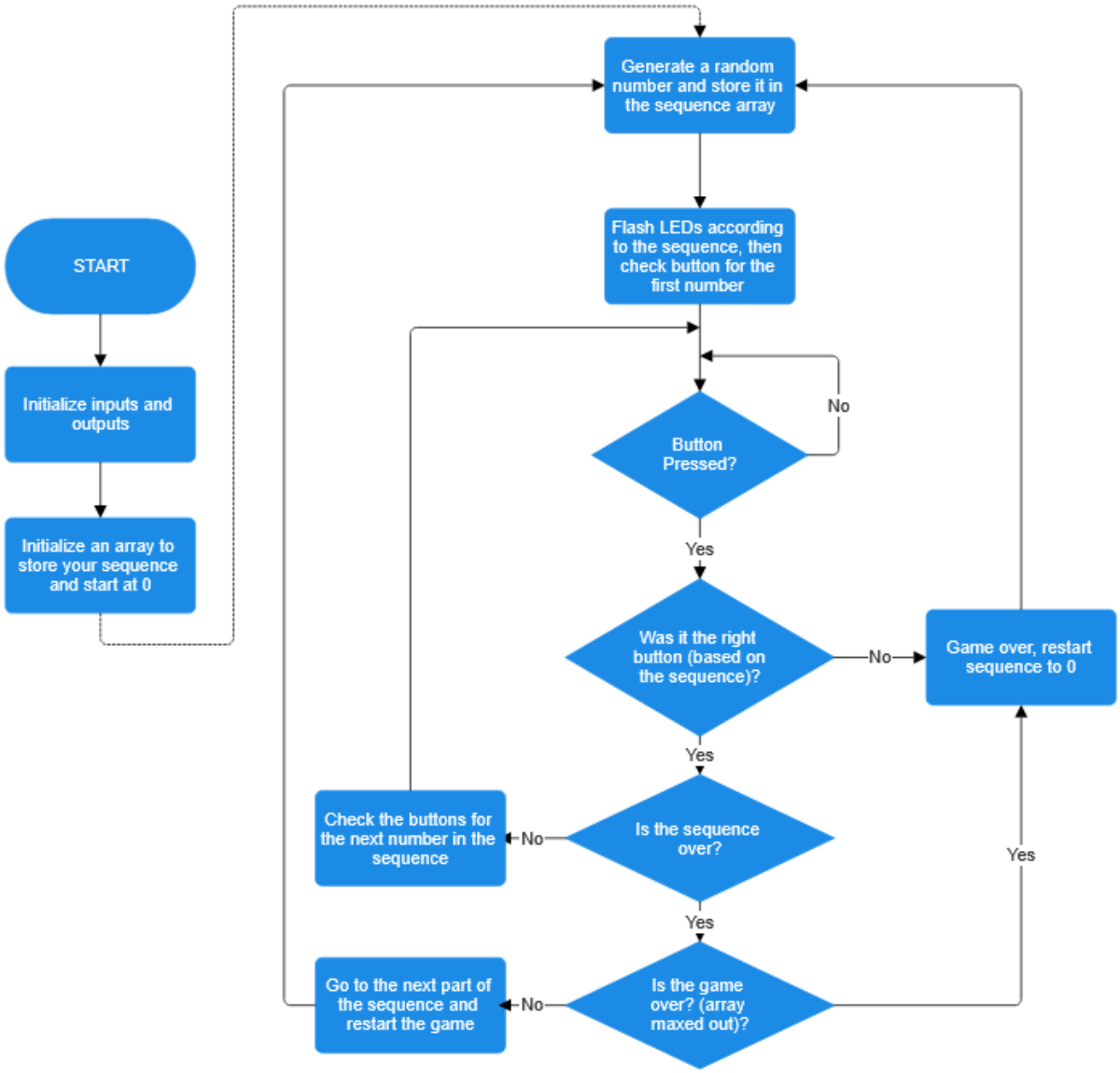
2. **Place and Wire the Pushbuttons**
   Insert the pushbuttons into the breadboard as shown. After, ensure that a ground connection from the lower bus strip is made to each pushbutton. Lastly, wire each of the pushbuttons to their respective digital pin on the Arduino.

3. **Place and Wire the LEDs and Resistors**
   Place an LED above each pushbutton on the breadboard, paying close attention so as <u>not</u> to place the LED's pins on the same row as a pushbutton's pins (there should be no electrical contact between these components). After placing the LEDs, wire a 330Ω resistor from the LED's cathode pin (the shorter leg, see image below) to the upper ground bus strip. The anode pin on each LED should be connected to its respective digital pin on the Arduino.

https://blogger.googleusercontent.com/img/b/R29vZ2xl/

Section 4
Coding

Use this flowchart to guide you while making your program:



Code Objectives

The following objectives serve as guidelines to help you complete the project in a way that guarantees success.

1) Generate a random sequence to illuminate the LEDs (the sequence length should start at 1 and increment by 1 with every successful round). Note that a brand new sequence should <u>not</u> be generated each round – you are only <u>adding</u> a random step to it after each round.
2) Flash the LEDs in the generated sequence (500ms ON and 300ms OFF between flashes)
3) Using buttons as user input, illuminate the corresponding LED each time a button is pressed (200ms ON with each press)
4) Capture button presses and check against generated sequence
    a. If the wrong button is pressed, immediately flash all four LEDs (200ms ON, 200ms OFF – repeat 4 times)
    b. If all the button presses are correct, move onto the next round
5) Each time a new round begins, display the round number in the serial monitor (ex: Round 1, Round 2, etc.)

With this information and the guidance on the following pages, you will be tasked with filling in the blanks within the provided sketch. Everywhere you see a line with a red marker next to it (see screenshot below), you are expected to write some code. There will also be a comment (text in grey) that explains what is missing on that line.

```
19  void setup() {
20    // *** Initialize buttons ***
21
22    // *** Initialize LEDs ***
23
```

Note that although only one line may be marked, some solutions require more than one line of code to be implemented – there is no wrong or right answer as long as everything works as intended!

Arrays

If you aren't already familiar, an array is a type of variable that allows you to store multiple values of the same data type under a single name. Let's examine the example below:

```
const int led[4] = {6, 7, 8, 9};
```

The `const int` portion (which isn't array-specific) is the datatype for this array. The array itself is defined as `led[4]`, where `led` is the array's name and `[4]` is it's size (basically how many values it can store). The actual data stored is in the form of a comma separated string, enclosed by curly brackets: `{6, 7, 8, 9}`.

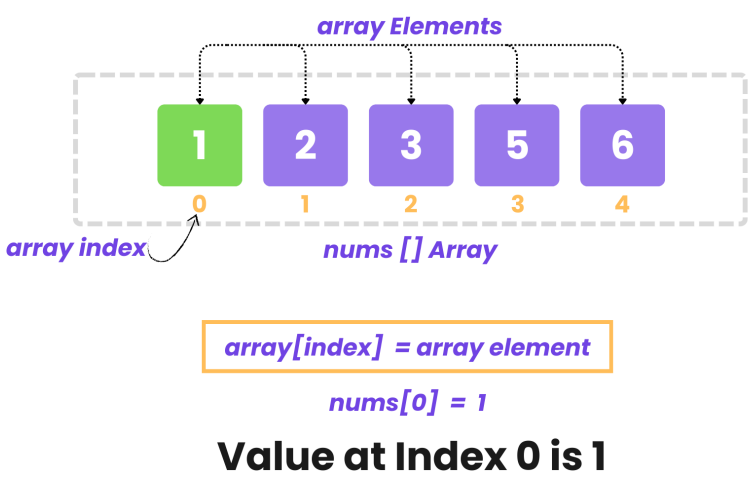
A Practical Example

Let's say we wanted to set a standard integer variable to the second value stored in the array from the previous example. To do this, we can write:

```
int value = led[1];
```

The reason why 1 is written within the square brackets as opposed to a 2 (for the second value) is because array indexing always begins at 0. This is a very important concept and means that the maximum value you would be able to call upon from this specific array is 3.

If you attempt to call upon a value outside the bounds of an array, this leads to serious errors that can lead to memory corruption – so, we generally try to watch out when referencing arrays.



https://read.learnyard.com/content/images/2025/04/index-of--the-given-array.png

Initialization

We must first set the `pinMode()` of the buttons and LEDs and ensure that all LEDs are off to begin with (since this code is being placed within `void setup()`, it will only run one time at the very start of the program). In other words, we must explicitly define whether these components are to be treated as *inputs* (the Arduino will read from the pin) or as *outputs* (the Arduino will set the pin to a value).

```
void setup() {
  // *** Initialize buttons ***

  // *** Initialize LEDs ***
```

***Hint**: The buttons require a special `pinMode()` that utilizes internal pullup resistors within the Arduino.*

- *Read about the different pinMode() options*

***Hint**: This process can, optionally, be greatly simplified using a `for` loop (for both the buttons and LEDs initialization), because the pin mappings for the components have already been defined within arrays.*

- *Read about for loops*

Displaying the Round Number

We now enter `void loop()`, where the program will continuously run the code contained within the section. At the start of each round, it would be helpful to indicate to the player which round they are on. To do this, you will need to find a way to display the round number.

The serial monitor (which can be opened by going to *Tools* > *Serial Monitor*) will allow you to view this info as the game progresses. Ensure to select a baud rate of 115200 once the window opens (in the upper right of the serial monitor window). To print the text, you can use the `Serial.print("")` function, where the actual text you want to print is contained within the quotation marks. However, this will only print text. To print the value of a variable (like the round number), remove the quotation marks and insert the variable name within the parenthesis. So, you'll have two lines - one that prints text, and the other that prints the round number after the text. Feel free to experiment with this function to properly understand it!

To add the brief delay (which can be a couple seconds, up to you), use the `delay()` function, where the number you insert within the parenthesis is in milliseconds.

```
void loop() {
  // *** Print current round number and wait brief delay ***
```

Generating a Random Number

We need to generate a random number to be displayed on any of the four LEDs. A simple function called `random()` can be used to accomplish this. Keep in mind that the random number it will generate corresponds to a value of the `led[]` array.

```
gamePattern[roundCount - 1] = /* *** Generate a random number *** */ ;
```

***Hint**: Remember that array indexing always begins at 0, and not 1.*

***Hint**: The second value of the function is excluded, so the range will be from the first number to the second number minus 1.*

- *Read about the random() function*

Flashing the LEDs In Order

At the start of every round, this block of code will briefly illuminate the LEDs one-by-one, following the generated sequence. You are now tasked with finding a way to pull this off given the following structure as a basis:

```
for (int x = 0; x < roundCount; x++) {
  // *** Turn ON the LED stored in gamePattern[x] ***
  delay(?);
  // *** Turn OFF the LED stored in gamePattern[x] ***
  delay(?);
}
```

***Hint**: If `digitalWrite(led[2], HIGH)`, for example, causes the third pin in the LED array to illuminate, how can we incorporate the `gamePattern[x]` array (where x is the incrementing value from the `for` loop) to select a value from the LED array based on the value at position x from the `gamePattern[]` array?*

***Hint**: You can find the delay values required in the Coding Objectives*

Printing the "Game Over" Message

The same process from printing the round number can be applied here, only simpler as we only need to display text. Note that the purpose of `Serial.println()` is to insert a blank line after the *Game Over* message is displayed.

Flashing All LEDs Upon Incorrect Button Press

If the user enters an incorrect button press at any point during the capture window (when the program waits and analyzes the user's input), all the LEDs should flash 4 times. Before the game restarts. This process occurs after the *Game Over* message is sent through the serial monitor.

***Hint**: You will need to use a `for` loop to run through the ON/OFF toggling of the LEDs 4 times.*

***Hint**: You can, optionally, two nested `for` loops within the first to toggle the LEDs' states effectively.*

***Hint**: You can find the delay values required in the Coding Objectives*

Conditionally Incrementing the Round Count Variable

The last step to complete this Simon Says game is to increment the round count variable at the very end of `void loop()`. This will ensure that, if the user enters the correct sequence and the code continues executing sequentially until the end of `void loop()`, then the user will proceed to the next round once the code restarts from the beginning of the loop again.

However, it isn't as simple as incrementing the count variable alone. We must make this process conditional using an `if()` statement to ensure that the current round count is less than the max rounds value (as defined at the beginning of the sketch). This prevents the sketch from incrementing the round count value to one beyond the scope of max rounds, which will lead to programming attempting to access out of bounds values from our arrays (which, as previously outlined, is very bad).

Troubleshooting

If anything is not working correctly, firstly refer to the circuit schematic to ensure all connections have been properly made. Then, refer to the programming flowchart to make sure your program contains all elements shown in the flow and in that order.

If you are still having issues, don't be afraid to raise your hand or call over a mentor to double-check your work.